

Язык программирования C++

Лекция 19: Инициализация объектов, final, tuple, regex, анонимные функции, function, type erasure, многопоточное программирование

Никита Лисица

2026

- Мы знаем, что основной способ инициализировать объект в C++ – вызвать конструктор

Инициализация объектов

- Мы знаем, что основной способ инициализировать объект в C++ – вызвать конструктор
- На самом деле, всё несколько сложнее :)

Инициализация объектов

- Мы знаем, что основной способ инициализировать объект в C++ – вызвать конструктор
- На самом деле, всё несколько сложнее :)
- Default initialization: `Type x;`

Инициализация объектов

- Мы знаем, что основной способ инициализировать объект в C++ – вызвать конструктор
- На самом деле, всё несколько сложнее :)
- Default initialization: `Type x;`
- Value initialization: `Type x{};`

Инициализация объектов

- Мы знаем, что основной способ инициализировать объект в C++ – вызвать конструктор
- На самом деле, всё несколько сложнее :)
- Default initialization: `Type x;`
- Value initialization: `Type x{};`
- Direct initialization: `Type x(a, b);`

Инициализация объектов

- Мы знаем, что основной способ инициализировать объект в C++ – вызвать конструктор
- На самом деле, всё несколько сложнее :)
- Default initialization: `Type x;`
- Value initialization: `Type x{};`
- Direct initialization: `Type x(a, b);`
- Copy initialization: `Type x = a;`

Инициализация объектов

- Мы знаем, что основной способ инициализировать объект в C++ – вызвать конструктор
- На самом деле, всё несколько сложнее :)
- Default initialization: `Type x;`
- Value initialization: `Type x{};`
- Direct initialization: `Type x(a, b);`
- Copy initialization: `Type x = a;`
- List initialization: `Type x{a, b};` или `Type x = {a, b};`

Default vs value initialization

- Default initialization `Type x;` для классов вызывает конструктор без аргументов, для примитивных типов ничего не делает

Default vs value initialization

- Default initialization `Type x;` для классов вызывает конструктор без аргументов, для примитивных типов ничего не делает
- **NB:** если у класса нет конструктора, компилятор сам создаст конструктор, рекурсивно вызывающий конструкторы всех полей (но для примитивных типов они тоже ничего не делают)

Default vs value initialization

- Default initialization `Type x;` для классов вызывает конструктор без аргументов, для примитивных типов ничего не делает
- **NB:** если у класса нет конструктора, компилятор сам создаст конструктор, рекурсивно вызывающий конструкторы всех полей (но для примитивных типов они тоже ничего не делают)
- Value initialization `Type x{};` зануляет объект, потом для классов вызывает конструктор без аргументов

Default vs value initialization

```
// Нет конструктора без аргументов
// Компилятор сам создаст конструктор,
// который ничего не будет делать
// (trivial constructor)
struct point {
    int x, y;
};

// Default initialization
// Значение остаётся не проинициализированным
int i;
// Поля остаются не проинициализированы
point p;

// Value initialization
// Значение будет нулём
int i{};
// Значение будет нулём
point p{};
```

- Синтаксис `Type x = {a, b, c};` может сделать одну из нескольких вещей:

List initialization

- Синтаксис `Type x = {a, b, c};` может сделать одну из нескольких вещей:
- Вызвать конструктор `Type` от аргументов `(a, b, c)`

List initialization

- Синтаксис `Type x = {a, b, c};` может сделать одну из нескольких вещей:
- Вызвать конструктор `Type` от аргументов `(a, b, c)`
- Если у `Type` нет конструктора, проинициализировать все поля значениями из `{a, b, c}` по порядку

List initialization

- Синтаксис `Type x = {a, b, c};` может сделать одну из нескольких вещей:
- Вызвать конструктор `Type` от аргументов `(a, b, c)`
- Если у `Type` нет конструктора, проинициализировать все поля значениями из `{a, b, c}` по порядку
- Если `Type` это массив, проинициализировать его элементы значениями из `{a, b, c}` по порядку

List initialization

- Синтаксис `Type x = {a, b, c};` может сделать одну из нескольких вещей:
- Вызвать конструктор `Type` от аргументов `(a, b, c)`
- Если у `Type` нет конструктора, проинициализировать все поля значениями из `{a, b, c}` по порядку
- Если `Type` это массив, проинициализировать его элементы значениями из `{a, b, c}` по порядку
- Вызвать конструктор от `std::initializer_list`

List initialization: примеры

```
struct point {  
    int x, y;  
    point(int x, int y)  
        : x(x), y(y)  
    {}  
};  
  
// Вызов конструктора point::point(int, int)  
point p = {1, 2};
```

List initialization: примеры

```
struct point {  
    int x, y;  
    point(int x, int y)  
        : x(x), y(y)  
    {}  
};  
  
// Вызов конструктора point::point(int, int)  
point p = {1, 2};
```

```
struct point {  
    int x, y;  
};  
  
// Aggregate initialization  
// Эквивалентно p.x = 1, p.y = 2  
point p = {1, 2};
```

List initialization: примеры

- При aggregate initialization можно указывать явно имена полей (*designated initializers*)

```
struct point {  
    int x, y;  
};  
  
point p = {.x = 1, .y = 2};
```

List initialization: примеры

- При aggregate initialization можно указывать явно имена полей (*designated initializers*)

```
struct point {  
    int x, y;  
};  
  
point p = {.x = 1, .y = 2};
```

```
// Инициализация массива  
int array[3] = {1, 2, 3};
```

Инициализация полей значением по умолчанию

- Поля структуры/класса можно инициализировать прямо при объявлении значением по умолчанию:

```
class Array {  
private:  
    int* data = nullptr;  
    size_t size = 0;  
};
```

Инициализация полей значением по умолчанию

- Поля структуры/класса можно инициализировать прямо при объявлении значением по умолчанию:

```
class Array {  
private:  
    int* data = nullptr;  
    size_t size = 0;  
};
```

- Эта инициализация автоматически встроится в любой конструктор, который явно не инициализирует эти поля сам

Инициализация полей значением по умолчанию

- Поля с таким значением по умолчанию можно не инициализировать при list/designated initialization:

```
struct point {  
    int id = 0;  
    int x, y;  
};  
  
// .id = 0 будет автоматически  
point p{.x = 1, .y = 2};
```


- `std::initializer_list<T>` – специальный шаблонный класс в стандартной библиотеке для инициализации контейнеров

- `std::initializer_list<T>` – специальный шаблонный класс в стандартной библиотеке для инициализации контейнеров
- Обычно ссылается на массив элементов `(T* data, size_t size)`, где-то созданных компилятором

- `std::initializer_list<T>` – специальный шаблонный класс в стандартной библиотеке для инициализации контейнеров
- Обычно ссылается на массив элементов `(T* data, size_t size)`, где-то созданных компилятором

```
// Вызывает конструктор от
// std::initializer_list<int>
std::vector<int> vec = {1, 2, 3, 4};
```

- `std::initializer_list<T>` – специальный шаблонный класс в стандартной библиотеке для инициализации контейнеров
- Обычно ссылается на массив элементов `(T* data, size_t size)`, где-то созданных компилятором

```
// Вызывает конструктор от  
// std::initializer_list<int>  
std::vector<int> vec = {1, 2, 3, 4};
```

- Все стандартные контейнеры имеют такие конструкторы

- Можно создать такой конструктор и своему классу:

```
class my_container {  
public:  
    my_container(std::initializer_list<int> l) {  
        // l.begin(), l.end() -- указатели на начало/конец  
        // l.data(), l.size(), l.empty() -- как у std::vector  
    }  
};  
  
my_container c = {1, 2, 3, 4};
```

- Можно запретить наследоваться от класса:

```
class A final {  
    // ...  
};  
  
// Ошибка компиляции: от A нельзя  
// наследоваться  
class B : public A {};
```

- Можно запретить переопределять виртуальный метод:

```
class A {  
public:  
    virtual void foo();  
};  
  
class B : public A {  
public:  
    void foo() final;  
};  
  
// Ошибка компиляции: foo() нельзя  
// переопределять  
class C : public B {  
public:  
    void foo() override;  
};
```

- Зачем использовать `final`?

- Зачем использовать **final**? Для оптимизации и для защиты от ошибок

```
class A {  
public:  
    virtual void foo();  
};  
  
class B : public A {  
public:  
    void foo() final;  
};  
  
void call_foo(B* ptr) {  
    // Можно не использовать dynamic dispatch:  
    // мы знаем, что это именно B::foo(), так  
    // как её нельзя переопределить  
    ptr->foo();  
}
```

- `std::tuple` – реализация *кортежа*, т.е. упорядоченного набора произвольных элементов

- `std::tuple` – реализация *кортежа*, т.е. упорядоченного набора произвольных элементов
- **NB:** во многих языках `tuple` – встроенная конструкция, но в C++ она реализована в стандартной библиотеке

std::tuple

- `std::tuple` – реализация *кортежа*, т.е. упорядоченного набора произвольных элементов
- **NB:** во многих языках tuple – встроенная конструкция, но в C++ она реализована в стандартной библиотеке
- Это шаблонный класс, параметризующийся хранимыми типами, например `std::tuple<int, float, std::string>`

std::tuple

- `std::tuple` – реализация *кортежа*, т.е. упорядоченного набора произвольных элементов
- **NB:** во многих языках `tuple` – встроенная конструкция, но в C++ она реализована в стандартной библиотеке
- Это шаблонный класс, параметризующийся хранимыми типами, например `std::tuple<int, float, std::string>`

```
std::tuple<int, float> tup1{15, 3.14f};

// Используем вывод шаблонных параметров классов (CTAD)
// -> std::tuple<char, double>
std::tuple tup2{'a', 10.0};

// Получить значение по compile-time индексу:
std::get<0>(tup1); // == 15
std::get<1>(tup2); // == 10.0
```

- Основное применение `std::tuple` – сохранить передать неизвестный набор значений куда-либо, обычно в шаблонном коде

std::tuple

- Основное применение `std::tuple` – сохранить передать неизвестный набор значений куда-либо, обычно в шаблонном коде
- Лучше не использовать `std::tuple` в нормальных интерфейсах:

```
// Что такое int и float?  
std::tuple<int, float> get_time();  
  
// Лучше иметь структуры с явными  
// именами полей:  
struct time {  
    int seconds;  
    float milliseconds;  
};  
  
time get_time();
```

- При работе со стандартной библиотекой часто приходится написать функцию/функциональный объект: компаратор для `std::sort` или `std::map`, предикат для `std::find_if`, и т.п.

Анонимные функции

- При работе со стандартной библиотекой часто приходится написать функцию/функциональный объект: компаратор для `std::sort` или `std::map`, предикат для `std::find_if`, и т.п.

```
struct less_by_first_character
{
    bool operator()(
        const std::string& s1,
        const std::string& s2) {
        return s1[0] < s2[0];
    }
};

std::sort(lines.begin(), lines.end(),
    less_by_first_character());
```

Анонимные функции

- При работе со стандартной библиотекой часто приходится написать функцию/функциональный объект: компаратор для `std::sort` или `std::map`, предикат для `std::find_if`, и т.п.

```
struct less_by_first_character
{
    bool operator()(
        const std::string& s1,
        const std::string& s2) {
        return s1[0] < s2[0];
    }
};

std::sort(lines.begin(), lines.end(),
    less_by_first_character());
```

- Писать целый класс неудобно: много лишнего кода (т.н. *boilerplate*), логика ‘размазана’ по коду

- Решение: *анонимные функции*, они же – *лямбда-функции* или просто *лямбды*

- Решение: *анонимные функции*, они же – *лямбда-функции* или просто *лямбды*
- Концепция пришла из *лямбда-исчисления* (Алонзо Чёрч, 1930-е) – базис для функционального программирования

- Решение: *анонимные функции*, они же – *лямбда-функции* или просто *лямбды*
- Концепция пришла из *лямбда-исчисления* (Алонзо Чёрч, 1930-е) – базис для функционального программирования
- $Y = \lambda f.(\lambda x.f(x\ x))\ (\lambda x.f(x\ x))$

Анонимные функции

- Решение: *анонимные функции*, они же – *лямбда-функции* или просто *лямбды*
- Концепция пришла из *лямбда-исчисления* (Алонзо Чёрч, 1930-е) – базис для функционального программирования
- $Y = \lambda f.(\lambda x.f(x\ x))\ (\lambda x.f(x\ x))$
- В императивных языках лямбда-функции – синтаксический сахар, т.е. удобный способ выразить часто повторяющуюся конструкцию

Анонимные функции

- Общий синтаксис:

```
[список захвата](аргументы){ тело функции; }
```

Анонимные функции

- Общий синтаксис:

[список захвата](аргументы){ тело функции; }

```
auto sqr = [](int x){ return x * x; };  
auto sum = [](int x, int y){ return x + y; };
```


Анонимные функции

- Общий синтаксис:

[список захвата](аргументы){ тело функции; }

```
auto sqr = [](int x){ return x * x; };  
auto sum = [](int x, int y){ return x + y; };
```

- Компилятор превратит лямбды в функциональные объекты:

```
struct sqr_t {  
    int operator()(int x) const { return x * x; };  
};  
  
struct sum_t {  
    int operator()(int x, int y) const { return x + y; };  
};  
  
auto sqr = sqr_t{};  
auto sum = sum_t{};
```

- Их удобно использовать как компараторы/предикаты:

```
// Отсортировать по возрастанию модуля
std::sort(array.begin(), array.end(),
    [](int x, int y){ return abs(x) < abs(y); });

// Найти элемент, кратный 8
std::find_if(array.begin(), array.end(),
    [](int x){ return (x % 8) == 0; });
```

Capture list

- Удобство функциональных объектов в том, что они могут *хранить данные* и использовать их в своём `operator()`:

```
struct less_than_ref {  
    int ref;  
    bool operator()(int x) const { return x < ref; }  
};  
  
std::partition(array.begin(), array.end(),  
    less_than_ref{15});
```

Capture list

- Удобство функциональных объектов в том, что они могут *хранить данные* и использовать их в своём `operator()`:

```
struct less_than_ref {  
    int ref;  
    bool operator()(int x) const { return x < ref; }  
};  
  
std::partition(array.begin(), array.end(),  
    less_than_ref{15});
```

- У лямбд для этого есть *список захвата (capture list)*:

```
int ref = 15;  
std::partition(array.begin(), array.end(),  
    [ref](int x){ return x < ref; });
```

Capture list

- 'Захватить' переменную в список захвата можно как по значению, так и по ссылке

Capture list

- ‘Захватить’ переменную в список захвата можно как по значению, так и по ссылке
- Захват по значению `[x](...){...}` создаёт копию переменной внутри лямбды

Capture list

- 'Захватить' переменную в список захвата можно как по значению, так и по ссылке
- Захват по значению `[x](...){...}` создаёт копию переменной внутри лямбды
- Захват по ссылке `[&x](...){...}` создаёт ссылку на переменную внутри лямбды

Capture list

- ‘Захватить’ переменную в список захвата можно как по значению, так и по ссылке
- Захват по значению `[x](...){...}` создаёт копию переменной внутри лямбды
- Захват по ссылке `[&x](...){...}` создаёт ссылку на переменную внутри лямбды
- **NB:** в захвате по ссылке выражение `&x` не означает указатель на переменную!

Capture list

- ‘Захватить’ переменную в список захвата можно как по значению, так и по ссылке
- Захват по значению `[x](...){...}` создаёт копию переменной внутри лямбды
- Захват по ссылке `[&x](...){...}` создаёт ссылку на переменную внутри лямбды
- **NB:** в захвате по ссылке выражение `&x` не означает указатель на переменную!

```
int count = 0;
int ref = 15;
std::partition(begin, end, [&count, ref](int x){
    ++count;
    return x < ref;
});
```

- Эквивалентный код без лямбд:

```
struct lambda_t {  
    int& count;  
    int ref;  
  
    bool operator()(int x) const {  
        ++count;  
        return x < ref;  
    }  
};  
  
int count = 0;  
int ref = 15;  
std::partition(begin, end, lambda_t{count, ref});
```

Capture list

- Элементы списка захвата не обязаны быть существующими переменными, но тогда их надо руками проинициализировать:

```
int ref = 10;  
auto func = [y = ref * 2](int x){ return x < y; };
```

Capture list

- Элементы списка захвата не обязаны быть существующими переменными, но тогда их надо руками проинициализировать:

```
int ref = 10;  
auto func = [y = ref * 2](int x){ return x < y; };
```

- По умолчанию, сгенерированный **operator()** помечен как **const** и не может менять значения в списке захвата

Capture list

- Элементы списка захвата не обязаны быть существующими переменными, но тогда их надо руками проинициализировать:

```
int ref = 10;  
auto func = [y = ref * 2](int x){ return x < y; };
```

- По умолчанию, сгенерированный `operator()` помечен как `const` и не может менять значения в списке захвата
- Это можно изменить ключевым словом `mutable`:

```
auto func = [count = 0](int x) mutable {  
    ++count;  
    return x < 15;  
};
```

Capture list

- Можно захватить всё, что используется в лямбде, автоматически по ссылке `[&]` или по значению `[=]`:

```
int ref = 10;

// Автоматический захват по значению
auto func1 = [=](int x){ return x < ref; };

// Автоматический захват по ссылке
auto func2 = [&](int x){ return x < ref; };
```

Capture list

- Можно захватить всё, что используется в лямбде, автоматически по ссылке `[&]` или по значению `[=]`:

```
int ref = 10;

// Автоматический захват по значению
auto func1 = [=](int x){ return x < ref; };

// Автоматический захват по ссылке
auto func2 = [&](int x){ return x < ref; };
```

- **NB:** Лучше не пользоваться автоматическим захватом, тяжело отследить битые ссылки

Анонимные функции: возвращаемое значение

- Тип возвращаемого значения у лямбд выводится автоматически

Анонимные функции: возвращаемое значение

- Тип возвращаемого значения у лямбд выводится автоматически
- Можно указать его явно, чтобы избежать ошибок или неявных преобразований:

```
auto sqr = [](int x) -> int {  
    return x * x;  
};  
  
auto cmp = [](int x, int y) -> bool {  
    return abs(x) < abs(y);  
};
```

- Лямбды можно использовать не только как предикаты или компараторы

- Лямбды можно использовать не только как предикаты или компараторы
- Настройка поведения сложного шаблонного кода

- Лямбды можно использовать не только как предикаты или компараторы
- Настройка поведения сложного шаблонного кода
- Callback'и (обработчики событий) в асинхронном коде (сеть, UI)

Анонимные функции: применение

- Лямбды можно использовать не только как предикаты или компараторы
- Настройка поведения сложного шаблонного кода
- Callback'и (обработчики событий) в асинхронном коде (сеть, UI)
- Библиотека **Boost.Hana** реализует метапрограммирование на лямбдах

- Если мы хотим куда-то принимать функцию или лямбду, у нас есть два варианта:

- Если мы хотим куда-то принимать функцию или лямбду, у нас есть два варианта:
 - Принимать указатель на функцию – тогда нельзя передать лямбду

- Если мы хотим куда-то принимать функцию или лямбду, у нас есть два варианта:
 - Принимать указатель на функцию – тогда нельзя передать лямбду
 - Сделать код шаблонным – тогда его придётся положить в заголовочный файл, это замедлит компиляцию, и его нельзя положить в динамическую библиотеку

- Если мы хотим куда-то принимать функцию или лямбду, у нас есть два варианта:
 - Принимать указатель на функцию – тогда нельзя передать лямбду
 - Сделать код шаблонным – тогда его придётся положить в заголовочный файл, это замедлит компиляцию, и его нельзя положить в динамическую библиотеку
- Решение: `std::function`

- `std::function<R(Args...)>` – специальный шаблонный класс, позволяющий хранить любой объект, который можно вызвать от списка аргументов `(Args...)`, и который возвращает тип `R`

- `std::function<R(Args...)>` – специальный шаблонный класс, позволяющий хранить любой объект, который можно вызвать от списка аргументов (`Args...`), и который возвращает тип `R`

```
std::function<int(int)> sqr = [](int x){  
    return x * x;  
};  
  
std::function<int(int, int)> sum = [](int x, int y){  
    return x + y;  
};
```

- `std::function` с конкретными параметрами – конкретный тип, его можно использовать в нешаблонных интерфейсах классов и библиотек

std::function

- `std::function` с конкретными параметрами – конкретный тип, его можно использовать в нешаблонных интерфейсах классов и библиотек

```
class UIController {
public:
    virtual onKeyDown(std::function<void(char)> callback) = 0;
    virtual onKeyUp (std::function<void(char)> callback) = 0;

    virtual onMouseMove
        (std::function<void(int, int)> callback) = 0;

    virtual ~UIController() {}
};

controller->onKeyDown([](char key){
    if (key == 'W') {
        moveForward();
    }
});
```

std::function: реализация

```
template <typename R, typename ... Args>
struct function_base {
    virtual R call(Args...) = 0;
};

template <typename R, typename ... Args, typename Object>
struct function_impl : base<R, Args...> {
    Object object;
    R call(Args... args) { return object(args...); }
};

template <typename R, typename ... Args>
struct function {
    template <typename Object>
    function(Object object)
        : impl(new function_impl<R, Args..., Object>(object))
    {}

    R operator()(Args... args) { return impl->call(args); }

private:
    function_base<R, Args...>* impl;
};
```

- `std::function` – обёртка для любого объекта, обладающего некоторым интерфейсом (`operator()` с нужными типами)

- `std::function` – обёртка для любого объекта, обладающего некоторым интерфейсом (`operator()` с нужными типами)
- Реализация использует настоящий интерфейс с виртуальными функциями

- `std::function` – обёртка для любого объекта, обладающего некоторым интерфейсом (`operator()` с нужными типами)
- Реализация использует настоящий интерфейс с виртуальными функциями
- Оборачивание конкретного объекта создаёт реализацию этого интерфейса, параметризующуюся типом хранимого объекта

- `std::function` – обёртка для любого объекта, обладающего некоторым интерфейсом (`operator()` с нужными типами)
- Реализация использует настоящий интерфейс с виртуальными функциями
- Оборачивание конкретного объекта создаёт реализацию этого интерфейса, параметризующуюся типом хранимого объекта
- Такой приём называется *type erasure* и часто используется чтобы разменять производительность на скорость компиляции и удобство архитектуры

- *Процесс* – отдельно запущенная программа со своим адресным пространством

- *Процесс* – отдельно запущенная программа со своим адресным пространством
- *Поток* – независимо выполняющаяся часть программы, использующая её адресное пространство

- *Процесс* – отдельно запущенная программа со своим адресным пространством
- *Поток* – независимо выполняющаяся часть программы, использующая её адресное пространство
- Разные *процессы* не могут просто так общаться через память (для этого нужны сокеты/файлы/shared memory)

- *Процесс* – отдельно запущенная программа со своим адресным пространством
- *Поток* – независимо выполняющаяся часть программы, использующая её адресное пространство
- Разные *процессы* не могут просто так общаться через память (для этого нужны сокеты/файлы/shared memory)
- Один *процесс* может иметь несколько *потоков*, и они все имеют общую память

- *Процесс* – отдельно запущенная программа со своим адресным пространством
- *Поток* – независимо выполняющаяся часть программы, использующая её адресное пространство
- Разные *процессы* не могут просто так общаться через память (для этого нужны сокеты/файлы/shared memory)
- Один *процесс* может иметь несколько *потоков*, и они все имеют общую память
- Единица исполнения кода – это *поток*

- В общем случае, потоков может быть больше, чем ядер CPU

Виды многозадачности

- В общем случае, потоков может быть больше, чем ядер CPU
- *Вытесняющая многозадачность* – ядро операционной системы выделяет кванты времени каждому потоку и переключается между ними на каждом ядре CPU

Виды многозадачности

- В общем случае, потоков может быть больше, чем ядер CPU
- *Вытесняющая многозадачность* – ядро операционной системы выделяет кванты времени каждому потоку и переключается между ними на каждом ядре CPU
- *Кооперативная многозадачность* – выполняющиеся потоки сами решают, когда отдать управление другому потоку

Виды многозадачности

- В общем случае, потоков может быть больше, чем ядер CPU
- *Вытесняющая многозадачность* – ядро операционной системы выделяет кванты времени каждому потоку и переключается между ними на каждом ядре CPU
- *Кооперативная многозадачность* – выполняющиеся потоки сами решают, когда отдать управление другому потоку
- В современных ОС реализована *вытесняющая многозадачность* на уровне потоков всех процессов системы

Виды многозадачности

- В общем случае, потоков может быть больше, чем ядер CPU
- *Вытесняющая многозадачность* – ядро операционной системы выделяет кванты времени каждому потоку и переключается между ними на каждом ядре CPU
- *Кооперативная многозадачность* – выполняющиеся потоки сами решают, когда отдать управление другому потоку
- В современных ОС реализована *вытесняющая многозадачность* на уровне потоков всех процессов системы
- Современные библиотеки работы с сетью (`node.js`, `asyncio`) асинхронные и используют *кооперативную многозадачность*, часто с помощью *корутин*

- При создании потока нужно

- При создании потока нужно
 - Выделить место под его стек

- При создании потока нужно
 - Выделить место под его стек
 - Выделить место под `thread_local` переменные

- При создании потока нужно
 - Выделить место под его стек
 - Выделить место под `thread_local` переменные
 - Проинициализировать информацию о потоке внутри ОС

- При создании потока нужно
 - Выделить место под его стек
 - Выделить место под `thread_local` переменные
 - Проинициализировать информацию о потоке внутри ОС
- При переключении потоков (т.н. *переключение контекста*, *context switch*) нужно

- При создании потока нужно
 - Выделить место под его стек
 - Выделить место под `thread_local` переменные
 - Проинициализировать информацию о потоке внутри ОС
- При переключении потоков (т.н. *переключение контекста*, *context switch*) нужно
 - Запомнить состояние всех регистров потока

- При создании потока нужно
 - Выделить место под его стек
 - Выделить место под `thread_local` переменные
 - Проинициализировать информацию о потоке внутри ОС
- При переключении потоков (т.н. *переключение контекста*, *context switch*) нужно
 - Запомнить состояние всех регистров потока
 - Загрузить состояние всех регистров нового потока

- При создании потока нужно
 - Выделить место под его стек
 - Выделить место под `thread_local` переменные
 - Проинициализировать информацию о потоке внутри ОС
- При переключении потоков (т.н. *переключение контекста*, *context switch*) нужно
 - Запомнить состояние всех регистров потока
 - Загрузить состояние всех регистров нового потока
 - Обновить MMU в CPU для нового процесса

- $\implies$ Создание и переключение потоков – дорогая операция

- $\implies$ Создание и переключение потоков – дорогая операция
- Перед использованием потоков нужно подумать, не будут ли накладные расходы выше, чем выигрыш от параллельности

- $\implies$ Создание и переключение потоков – дорогая операция
- Перед использованием потоков нужно подумать, не будут ли накладные расходы выше, чем выигрыш от параллельности
- Типичное применение:

- $\implies$ Создание и переключение потоков – дорогая операция
- Перед использованием потоков нужно подумать, не будут ли накладные расходы выше, чем выигрыш от параллельности
- Типичное применение:
 - Распараллеливание алгоритмов (умножение матриц, сортировка)

- $\implies$ Создание и переключение потоков – дорогая операция
- Перед использованием потоков нужно подумать, не будут ли накладные расходы выше, чем выигрыш от параллельности
- Типичное применение:
 - Распараллеливание алгоритмов (умножение матриц, сортировка)
 - Real-time приложения (UI в одном потоке, сложные операции в другом)

- $\implies$ Создание и переключение потоков – дорогая операция
- Перед использованием потоков нужно подумать, не будут ли накладные расходы выше, чем выигрыш от параллельности
- Типичное применение:
 - Распараллеливание алгоритмов (умножение матриц, сортировка)
 - Real-time приложения (UI в одном потоке, сложные операции в другом)
 - Сетевые приложения (несколько потоков, каждый обрабатывает свой запрос)

- Стандартный класс `std::thread` реализует отдельный поток выполнения

- Стандартный класс `std::thread` реализует отдельный поток выполнения
- Конструктор создаёт и сразу запускает поток

- Стандартный класс `std::thread` реализует отдельный поток выполнения
- Конструктор создаёт и сразу запускает поток
- Метод `join()` ждёт, пока поток закончит выполняться

- Стандартный класс `std::thread` реализует отдельный поток выполнения
- Конструктор создаёт и сразу запускает поток
- Метод `join()` ждёт, пока поток закончит выполняться

```
void my_thread_function();  
  
std::thread th(&my_thread_function);  
  
// Можем выполнять другие действия, пока работает поток  
...  
  
th.join()
```

- Стандартный класс `std::thread` реализует отдельный поток выполнения
- Конструктор создаёт и сразу запускает поток
- Метод `join()` ждёт, пока поток закончит выполняться

```
void my_thread_function();

std::thread th(&my_thread_function);

// Можем выполнять другие действия, пока работает поток
...

th.join()
```

- **NB:** с потоком, вызвавшим функцию `main`, ничего сделать нельзя!

- В конструктор потока можно передать и лямбду

```
// Лямбда без аргументов - можно опустить круглые скобки
std::thread th([]{
    // Тело функции, которая будет исполняться
    // в отдельном потоке
    ...
});
```


- В конструктор потока можно передать и лямбду

```
// Лямбда без аргументов - можно опустить круглые скобки
std::thread th([]{
    // Тело функции, которая будет исполняться
    // в отдельном потоке
    ...
});
```

- В конструктор потока можно передать аргументы для вызываемой функции:

```
void thread_func(int id);

// Создаст поток и вызовет в нём
// thread_func(10)
std::thread th(&thread_func, 10);
```

- Пример: параллельное сложение двух векторов

```
void sum(const int* src1, const int* src2,
         int* dst, int count)
{
    for (int i = 0; i < count; ++i)
        dst[i] = src1[i] + src2[i];
}

void parallel_sum(const int* src1, const int* src2,
                 int* dst, int count)
{
    // Тут хорошо бы округлить указатель до размера кеш-линии,
    // см. std::hardware_destructive_interference_size
    int half = count / 2;
    std::thread th1(&sum, src1, src2, dst, half);
    std::thread th2(&sum, src1 + half, src2 + half,
                  dst + half, count - half);
    th1.join();
    th2.join();
}
```

- **NB:** начиная с C++17 можно такие вещи не реализовывать руками:

```
void parallel_sum(const int* src1, const int* src2,
                 int* dst, int count)
{
    // Вместо лямбды можно std::plus<int>{}
    std::transform(std::execution::par, src1, src1 + count,
                  src2, dst, [](int x, int y){ return x + y; });
}
```